AI-Based Intelligent Video Analytics Platform for Border Surveillance

Using Existing CCTV Infrastructure

Problem Statement	SIH-26187
Organisation	Ministry of Defence / Border Security Force
Category	Software
Team Name	RootError
Domain	Computer Vision · AI/ML · Edge Computing
Date	September 09, 2026

Table of Contents

Chapter 1 — Executive Summary

Border surveillance in India's strategic zones relies heavily on legacy CCTV networks that demand continuous, round-the-clock human monitoring. This operational paradigm introduces critical failure points rooted in human error, cognitive fatigue, and inconsistent interpretation — all of which are unacceptable in high-stakes security environments.

Team **RootError** proposes **SIH-26187**: an AI-powered Intelligent Video Analytics Platform that retrofits existing CCTV infrastructure with a real-time, multi-module computer vision pipeline. The system automates troop verification, vehicle identification and number plate recognition, suspicious activity detection, and virtual fence intrusion alerting — while maintaining a robust audit trail independent of any human operator.

The solution is designed to operate on commodity GPU hardware, using alternating-frame inference strategies to maximise throughput while minimising computational load.

Chapter 2 — Problem Statement & Background

Traditional border surveillance workflows are characterised by the following sequential, human-dependent steps:

CCTV Cameras (passive video capture)

▼

Continuous Human Monitoring

▼

Manual Observation & Interpretation

▼

Manual Identification (troop/vehicle/threat)

▼

Manual Logging (paper or digital entry)

▼

Manual Alerting (radio / phone call)

▼

Delayed / Human-Dependent Response

This workflow is inherently vulnerable to human error, attention lapses during long shifts, lack of standardised logging, and delayed incident response. The parameters that must be continuously tracked include:

- **Troop Positioning & Badge Verification** — Ensuring authorised personnel are in designated zones.
- **Vehicle Passthroughs & Internal Movement** — Tracking vehicles at checkposts and within restricted bases.
- **Number Plate Recording** — Logging all vehicle movement through gates and outposts.
- **Suspicious Behavioural Detection** — Identifying anomalous human patterns (psychological/behavioural signals).
- **Virtual Fence Intrusion Detection** — Instant alerts when any entity crosses a defined virtual boundary.

Goal	Replace the manual surveillance pipeline with an automated, AI-driven system that is auditable, real-time, and independent of human attention.

Chapter 3 — Proposed System Architecture

The proposed platform is a **hybrid rule-based + ML inference system** integrated with a centralised database management layer. Video feeds are ingested via RTSP/IP protocols and distributed across three concurrent processing pipelines using an alternating-frame strategy.

High-Level Architecture

■ RTSP / IP Camera Video Feed Ingestion

▼

■ Frame Demultiplexer (nth, nth+1, nth+2 routing)

▼

■ Module A: Human + Face Detection & Troop Verification

▼

■ Module B: Vehicle Detection, Classification & ANPR

▼

■ Module C: Virtual Fence Intrusion Detection & Alerting

▼

■ Rule Engine + ML Inference Aggregator

▼

■ Audit Logger & Event Database

▼

■ Real-Time Alert Management System

▼

■ Operator Dashboard (Read-Only Verification Layer)

Design Principles

- **Non-invasive deployment:** Works with existing IP/RTSP-compatible CCTV cameras — no hardware replacement required.
- **Alternate-frame inference:** Each pipeline processes every 3rd frame, tripling effective throughput with the same GPU budget.

- **Human-out-of-loop automation:** All decisions (identification, flagging, alerting) are machine-generated and logged atomically.
- **Modular architecture:** Each detection module is independently deployable and upgradeable.
- **Edge-friendly:** Designed to run on mid-range NVIDIA GPUs (RTX 3060 class or equivalent embedded hardware).

Chapter 4 — Module 1 — Human Detection & Troop Verification

This module processes every **nth frame** from each camera feed to detect human presence, identify personnel through facial recognition, and cross-reference detected positions against the authorised patrol schedule issued by the Post/Platoon Commander.

4.1 Human & Face Detection Pipeline

Incoming Frame (nth) from Camera Feed

▼

YOLOv8 — Full-Body Human Detection

▼

ROI Crop → Face Detection (RetinaFace / MTCNN)

▼

Face Embedding Extraction (ArcFace / FaceNet)

▼

Vector Similarity Search → Personnel Database

▼

Position Tagged with Identity + Confidence Score

▼

Cross-Reference: Authorised Zone & Schedule

▼

■ Verified / ■ Unauthorised Presence Alert

4.2 Troop Positioning Verification

Each patrol shift is pre-loaded by the Platoon Commander into the system as a **Stationed Allotment Record (SAR)**. The system continuously checks whether identified personnel are within their designated patrol zones using camera-zone mapping.

- If a soldier is detected outside their assigned zone → **Zone Violation Alert**
- If a face is unrecognised → **Unknown Personnel Alert**
- If a scheduled post is unmanned beyond a threshold duration → **Post Abandonment Alert**

4.3 Badge Verification (Supplementary)

In scenarios where facial recognition is obstructed (helmets, masks, night conditions), a secondary OCR-based badge/ID number detection is employed. The badge bounding box is detected, cropped, upscaled and passed through EasyOCR to extract the service number, which is then validated against the personnel database.

Models Used	YOLOv8n (human detection) | RetinaFace / MTCNN (face detection) | ArcFace / FaceNet (face embedding) | EasyOCR (badge OCR) | FAISS (vector similarity search)

Chapter 5 — Module 2 — Vehicle Detection, Classification & ANPR

Every **(n+1)th frame** is routed to the vehicle processing pipeline. The module performs two sequential tasks: (1) vehicle detection and military type classification, and (2) Automatic Number Plate Recognition (ANPR) followed by database verification.

5.1 Vehicle Detection & Military Classification

A YOLOv8n model is first used as a general vehicle detector (pre-trained on COCO dataset). Detected vehicle bounding boxes are then passed to a fine-tuned classifier trained specifically on military vehicle types:

Vehicle Type	Examples	Training Samples	Action on Detection
Combat	Tanks, APCs, IFVs	300 images	Flagged if unregistered
Transport	Trucks, Supply Lorries, Ambulances	300 images	Logged & verified
Patrol	Jeeps, Motorcycles, Mini-Trucks	300 images	Position cross-checked

Training data is split in a **70-20-10** ratio (train/validation/test). Transfer learning from the COCO pre-trained backbone significantly reduces the training data requirement.

5.2 Automatic Number Plate Recognition (ANPR)

Vehicle Detected → Bounding Box Extracted

▼

Number Plate Region Detection (YOLOv8 / Roboflow Pre-trained)

▼

Plate ROI Crop → Perspective Correction (Homography)

▼

Image Upscaling (Bicubic / Lanczos Interpolation)

▼

OCR Engine: PaddleOCR / EasyOCR (CUDA-accelerated)

▼

Extracted Registration Number

▼

API Call → Military Vehicle Registry Database

▼

Cross-Match: Detected Type vs. Registered Type

▼

■ Verified Tag / ■■ Mismatch Flag / ■ Unknown Plate Alert

5.3 Plate Extraction Technique

The YOLO model outputs four bounding coordinates (x1, y1, x2, y2) for the number plate region. A perspective transformation (homography matrix) is applied to deskew and flatten the plate, compensating for camera angle. The cropped plate is then upscaled using Lanczos interpolation to enhance character legibility before passing to the OCR engine.

- OCR tools: **PaddleOCR** (CUDA-optimised, high accuracy on alphanumeric plates), **EasyOCR** (fallback, broader language support)
- Verification outcome types: **Verified**, **Type Mismatch** (vehicle type doesn't match registry), **Unknown Plate** (not in database)
- All vehicle events are written to the **Movement Log Database** with timestamp, camera ID, plate number, detected type, and verification status

Chapter 6 — Module 3 — Virtual Fencing & Intrusion Detection

The Virtual Fencing module processes every **(n+2)th frame** and implements a geo-referenced, polygon-based perimeter system that triggers automated alerts the instant any human or vehicle entity crosses a pre-configured virtual boundary — without requiring any physical fence or sensor modification.

6.1 What is Virtual Fencing?

A virtual fence is a **software-defined boundary** drawn as one or more polygons over the camera's field of view. These polygons define restricted zones, buffer zones, and entry/exit corridors. The system continuously monitors all detected entities and evaluates their spatial relationship to these polygons in real time.

Key Principle	A virtual fence does not prevent intrusion — it provides instantaneous, automated, machine-generated detection and alerting, removing the dependency on a human operator noticing the event.

6.2 Zone Configuration

The system supports multiple concurrent zone types per camera, all configurable via the operator dashboard:

Zone Type	Definition	Trigger Condition	Alert Colour
Red Zone (Restricted)	No authorised entry at any time	Immediate HIGH-priority alert on any detection	Red
Amber Zone (Buffer)	Entry only during authorised windows	Alert if entered outside permitted hours	Amber
Green Zone (Monitored)	Normal operational area	Logged; alert only on loitering > threshold	Green
Corridor (Directional)	Defined entry/exit path	Alert on reverse traversal or prolonged dwell	Teal

6.3 Intrusion Detection Pipeline

Incoming Frame (n+2) from Camera Feed

▼

Multi-Class Object Detection: Human + Vehicle (YOLOv8)

▼

Object Tracking: ByteTrack / DeepSORT (persistent IDs across frames)

▼

Bounding Box Centroid Extraction for each tracked entity

▼

Point-in-Polygon Test (Ray Casting Algorithm)

▼

Zone Classification: Safe / Buffer / Restricted

▼

Dwell Timer Update (continuous frames in zone)

▼

Threshold Evaluation: Instant Cross / Loitering / Rapid Exit

▼

■ Alert Generated → Alert Manager → Push Notification / Siren / Dashboard

▼

■ Event Logged: Entity ID, Zone, Timestamp, Camera ID, Snapshot

6.4 Point-in-Polygon (PiP) Algorithm

For each detected entity, the system computes the centroid of its bounding box. A **Ray Casting Algorithm** then determines whether this centroid lies inside any defined fence polygon:

- Cast a horizontal ray from the centroid point towards +∞
- Count the number of polygon edge intersections
- **Odd intersections** → point is *inside* the polygon (intrusion detected)
- **Even intersections** → point is *outside* the polygon (safe)

This O(n) per entity computation is extremely lightweight and runs on CPU, leaving GPU resources exclusively for the neural network inference pipeline.

6.5 Object Tracking for Persistent Intrusion Monitoring

Detection alone is insufficient for virtual fencing — an entity that appears for a single frame should not necessarily trigger a high-priority alert. The system uses **ByteTrack** (and optionally DeepSORT as a fallback) to maintain consistent track IDs across frames:

- **Track assignment:** Each detected entity receives a unique track_id that persists as long as the entity remains visible.
- **Zone entry event:** Logged at the first frame where the centroid crosses into a zone.
- **Dwell time:** Accumulated per track_id; triggers loitering alerts after a configurable threshold (e.g., 30 seconds in Amber Zone).
- **Zone exit event:** Logged when the centroid leaves the zone polygon; used for direction-of-travel analysis.
- **Track loss handling:** If a track is lost (entity leaves frame or occlusion), the dwell timer is frozen for a configurable grace period before the track is retired.

6.6 Alert Response Matrix

Event	Trigger Timing	Priority	Response Actions
Human in Red Zone	Immediate	HIGH	Push + Siren + Dashboard flash
Vehicle in Red Zone	Immediate	HIGH	Push + Siren + Gate Lock Signal
Human in Amber Zone (out of hours)	5s dwell	MEDIUM	Push + Dashboard
Loitering > 30s (Green)	30s dwell	LOW	Dashboard log + optional push
Reverse corridor traverse	Immediate	MEDIUM	Push + Dashboard
Unknown entity in any zone	Immediate	HIGH	Push + Siren + Snapshot capture

Chapter 7 — Alternate Frame Processing Strategy

Running three separate deep-learning pipelines simultaneously on every frame would require significant GPU memory and compute. The alternate-frame strategy distributes the inference load across three interleaved pipelines:

Frame Slot	Pipeline Assigned
Frame n (0, 3, 6, …)	Human Detection + Facial Recognition + Troop Verification
Frame n+1 (1, 4, 7, …)	Vehicle Detection + Classification + ANPR
Frame n+2 (2, 5, 8, …)	Virtual Fence Intrusion Detection + Object Tracking

- **Effective detection rate:** At 30 fps, each pipeline processes 10 frames/second — sufficient for typical security scenarios where events unfold over multiple seconds.
- **GPU budget:** Peak VRAM usage is bounded by the heaviest single pipeline, not the sum of all three.
- **Configurable stride:** The frame offset n can be tuned based on available hardware. On higher-end GPUs, n can be reduced to process every other frame per pipeline.

Chapter 8 — Technology Stack

Layer	Technology	Purpose
Video Ingestion	OpenCV (cv2.VideoCapture, RTSP)	Stream reading, frame demux
Object Detection	YOLOv8 (Ultralytics)	Human, vehicle, plate detection
Object Tracking	ByteTrack / DeepSORT	Persistent entity tracking
Face Recognition	RetinaFace + ArcFace	Face detection + embedding
OCR	PaddleOCR / EasyOCR (CUDA)	Number plate & badge text
Image Enhancement	OpenCV: Bicubic/Lanczos Interp.	Plate upscaling before OCR
Fencing Logic	NumPy + Shapely (PiP)	Polygon zone management
Backend API	FastAPI + Python	Internal service communication
Database	PostgreSQL + TimescaleDB	Event logs, time-series data
Vector Search	FAISS	Fast face embedding lookup
Alert Delivery	Firebase Cloud Messaging / MQTT	Push alerts to operators
Dashboard	HTML5 + Vanilla JS + WebSocket	Thin, low-latency operator interface
Backend Runtime	Node.js + TypeScript (Hono)	Central service orchestrator & API
AI Runtime	Python 3.12+ (FastAPI/uvicorn)	Native GPU-accelerated inference

Chapter 9 — System Workflow & Data Flow

The end-to-end data flow from raw video capture to actionable alert follows this sequence:

- **Step 1:** IP/RTSP cameras stream H.264/H.265 video to the Edge Inference Node
- **Step 2:** Stream Receiver decodes frames using OpenCV with hardware-accelerated NVDEC
- **Step 3:** Frame Demultiplexer assigns frames to Pipeline A, B, or C by frame index modulo 3
- **Step 4:** Each pipeline runs inference asynchronously on separate CUDA streams
- **Step 5:** Detection results are passed to the Event Aggregator with bounding boxes + confidence
- **Step 6:** Event Aggregator applies rule-based filters and cross-references with the DB
- **Step 7:** Flagged events are written atomically to the Audit Log Database
- **Step 8:** Alert Manager dispatches notifications via FCM / MQTT based on severity matrix
- **Step 9:** Operator Dashboard receives real-time event stream via WebSocket
- **Step 10:** Human operator reviews alerts (read-only) and can escalate or dismiss

Chapter 10 — Auditing, Logging & Alert Management

A core design requirement of SIH-26187 is that all system actions must be **fully auditable and independent of human intervention**. Every detection, classification, verification, and alert is persisted atomically to a tamper-evident event log.

Event Log Schema

Field	Type	Description
event_id	UUID	Unique event identifier
camera_id	VARCHAR	Source camera identifier
frame_index	BIGINT	Absolute frame number in stream
timestamp_utc	TIMESTAMPTZ	Event UTC timestamp (microsecond precision)
pipeline_module	ENUM	HUMAN | VEHICLE | VIRTUAL_FENCE
entity_type	ENUM	PERSON | VEHICLE | UNKNOWN
entity_id	VARCHAR	Recognised identity or plate number
confidence	FLOAT	Model confidence score (0.0 – 1.0)
zone_id	VARCHAR	Zone polygon ID (for fence events)
alert_level	ENUM	HIGH | MEDIUM | LOW | INFO
verified	BOOLEAN	Cross-reference verification result
snapshot_path	TEXT	Path to saved event frame snapshot

Chapter 11 — Deployment & Infrastructure

Component	Technology / Hardware	Rationale
Edge Node (per outpost)	NVIDIA Jetson AGX Orin / Server-grade GPU	Local inference; minimal latency
Central Server	On-premise rack server with NVIDIA A40/A100	Multi-camera aggregation, DB hosting
Network	Secure LAN / Optical fibre backbone	RTSP stream transport
OS	Windows 11 / Ubuntu Linux	Host OS with native CUDA drivers
Process Management	Native Process Runtime (Node.js + Python)	Direct hardware GPU access without virtualization o
Database	PostgreSQL 16	High-performance native relational & time-series sto

Chapter 12 — Future Enhancements & Scalability

- **Thermal Camera Integration:** Extend the pipeline to process IR/thermal feeds for night-time and all-weather intrusion detection without visible light.
- **Drone Detection Module:** Add a dedicated YOLO-based aerial object detection pipeline for UAV/drone identification above the fencing perimeter.
- **Behavioural Anomaly Detection:** Integrate an LSTM/Transformer-based model to flag non-standard movement patterns (running, erratic movement, formation changes).
- **Federated Learning:** Allow edge nodes to contribute to model improvement without transmitting raw video data off-site, preserving operational security.
- **Multi-Camera Stitching:** Fuse overlapping fields of view from adjacent cameras into a unified spatial map for tracking entities across camera boundaries.
- **Biometric Integration:** Interface with UIDAI/MoD biometric databases for civilian visitor management at checkposts.
- **Automated Reporting:** Scheduled daily/weekly PDF audit reports generated automatically from the event database for command review.

---

*This document was prepared by Team **RootError** for Smart India Hackathon 2026, Problem Statement SIH-26187. All technical details represent the team's proposed approach and are subject to revision based on further research and prototyping.*